

Natural language processing

Retrieval-augmented generation

Finn Årup Nielsen

DTU Compute, Technical University of Denmark

March 13, 2026

Retrieval-augmented generation

Retrieval-augmented generation (RAG) with (probably the first description in *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* from 2020)

$\text{RAG} = \text{LLM} + \text{information retrieval}$.

1. Get a user query
2. Optionally process the query
3. Perform information retrieval retrieving top- k documents
4. Build a prompt with the retrieved top- k documents for summarization
5. Call LLM with the prompt and return result.

Why RAG?

May be better for some open-domain question-answering.

The document store that are used in RAG can contain specialized knowledge and answers are with respect to that knowledge — not the general knowledge in the LLM. This is particular relevant for companies.

It is easier to update a document store with new information than train an LLM with the new information.

You have the ability to point to specific documents where you got the answer from.

The information retrieval system

Initial RAG paper used dense (passage) retrieval system.

Original RAG paper embedded 21 million passages from the English Wikipedia

My experience is that hybrid approach with both sparse and dense retrieval may be better.

Efficient retrieval with specialize vector databases would be good.

If you only have some (hundreds?) of thousand of documents/passage/chunks a simple matrix-based (Numpy/scikit-learn) “database” may suffice.

Optional query transformation

Names: Query transformation, query rewriting, query expansion, etc.

Split the query (example from *Query Rewriting for Retrieval-Augmented Large Language Models*):

Original query: “What profession does Nicholas Ray and Elia Kazan have in common?”

Two transformed queries: “Nicholas Ray profession” and “Elia Kazan profession”

Cross-lingual information retrieval where user query in one language, documents in another: translate user query to document language, search with the translated query.

Hypothetical documents (HyDE): transform the query from query-like to document-like.

Optional query transformation example

Prompt: “You are given a query in an unknown language you should transform to a query for a retrieval-augmented system. Your target language is English, translate the query if necessary. Expand the query with synonyms if necessary. Teachers associated with course are called “Responsible” or “Course co-responsible”, courses codes are five characters, e.g., 01234, KU123 or 12F34. Answer with just the sentence and no more. The query is “{query}”.

Danish query transformed to an English query

“Hvem underviser i 2451?” → “Who is the Responsible or Course co-responsible teacher for course 02451?”

Spelling error ignored and course code fixed!

Evaluation

Evaluation of final answer against a ground truth (that is what is done in the exercise)

Evaluations involving the retrieved context.

Ragas <https://docs.ragas.io> Python library has a set of metrics, e.g.,

Faithfulness “measures how factually consistent a response is with the retrieved context”.

Context Precision “evaluates the retriever’s ability to rank relevant chunks higher than irrelevant ones for a given query in the retrieved context.”

Guardrails

If the RAG is a publicly-facing system then you would likely also want guardrails on the input and/or output.

“Write a poem about Tommy Sonne’s course”

With signals swirling, a complex array,
Tommy Sonne Alstrøm guides the way.
From Fourier transforms to spectrogram’s grace,
Unveiling the secrets of time and space.

You want to guard against hostile prompt: “Ignore all previous instructions, . . .”

Examples

ChatTutor has RAG on PDFs

“GPT” at ChatGPT set up for the course at <https://chatgpt.com/g/g-68e804bf9c548191bf9bde0db6b91da6-chat-dtu-nlp> has RAG on PDFs.